

n8n - La Theorie

Cours complet pour debutant : les 5 lecons essentielles
Projet : automatisation meteo quotidienne par email

Ce document resume toute la partie theorique de n8n, organisee en 5 lecons. Chaque lecon contient l'explication des concepts, les mots-cles importants, et a la fin la question posee avec sa reponse correcte, pour reviser facilement si tu oublies.

Objectif du projet

Chaque jour a 8h00 -> appeler une API meteo sur internet -> extraire les informations (temperature, ville, description) -> les envoyer par email. Automatiquement, sans intervention.

Lecon 1 - C'est quoi n8n ?

L'idee principale : n8n est un outil qui fait des taches a ta place, automatiquement, pour que tu ne les fasses pas a la main.

Image mentale : une chaine de montage (assembly line). Quelque chose entre au debut, passe par plusieurs machines, chaque machine fait un seul travail, et a la fin on obtient un produit fini.

Le vocabulaire de base :

- **Workflow** = la chaine de montage complete (toute ton automatisation).
- **Node** = une machine = une seule etape = un seul travail. (Le mot le plus important.)
- **Connexions** = les lignes qui relient les nodes (le tapis roulant).
- **Data** (les donnees) = ce qui circule d'un node a l'autre.

Regle d'or : la data circule toujours de gauche a droite. Le premier node est a gauche, le dernier a droite. Chaque node recoit la data du node precedent, fait son travail, et passe le resultat au suivant.

Ton projet = une chaine de 4 machines :

- Machine 1 (Trigger) : se reveille chaque jour a 8h00 -> demarre la chaine.
- Machine 2 : va sur l'API meteo et demande la meteo -> recupere les donnees brutes.
- Machine 3 : garde seulement temperature, ville, description -> nettoie les donnees.
- Machine 4 : met tout dans un email et l'envoie -> produit fini.

Question : Dans ton projet meteo, quelle machine est le "trigger" et quel est son role ?

Reponse correcte : Le **Schedule Trigger**. Son role : a 8h00, il se declenche (fires) et demarre tout le workflow. C'est toujours le premier node, et chaque workflow a besoin d'un trigger sinon rien ne se passe.

Lecon 2 - Les types de nodes

Il existe 3 familles de nodes. Si tu comprends ces 3, tu comprends 90% de n8n.

1. Trigger nodes (les declencheurs) - "QUAND ca démarre ?"

- **Schedule Trigger** -> démarre a une heure précise (ton 8h00).
- **Webhook** -> démarre quand il recoit un message d'une autre application.
- **Manual Trigger** -> démarre quand TU cliques sur "Test" (utile pendant la construction).

2. Action nodes (les actions) - "FAIRE quelque chose"

- **HTTP Request** -> appelle n'importe quelle API sur internet (pour récupérer la météo).
- **Gmail / Send Email** -> envoie un email.
- **Google Sheets** -> écrit des données dans un tableur.

3. Logic / Data nodes - "DECIDER ou TRANSFORMER"

- **Edit Fields (Set)** -> garde/renomme seulement les données voulues. C'est l'étape "extraire".
- **IF** -> fait un choix : si il pleut -> faire X, sinon -> faire Y.
- **Filter, Merge, Loop** -> plus avancées.

Trigger	Quand ca démarre ?	Schedule Trigger
Action	Faire une tâche / API	HTTP Request, Send Email
Logic/Data	Decider / nettoyer	Edit Fields

Question : Le node qui appelle l'API météo : quelle famille, et quel nom ?

Reponse correcte : C'est une **Action**, et le node s'appelle **HTTP Request**. Son rôle : aller à l'adresse de l'API, demander les données, et les ramener.

Lecon 3 - A quoi ressemble la data : le JSON

Quand le node HTTP Request appelle l'API, la reponse arrive dans un format appele JSON. Toutes les API parlent JSON, il faut savoir le reconnaitre.

Le JSON = des paires "cle : valeur". Comme un formulaire :

```
{
  "city": "Tunis",
  "temperature": 28,
  "humidity": 45,
  "description": "clear sky"
}
```

- **cle** = le nom de l'info (a gauche) : city, temperature...
- **valeur** = l'info reelle (a droite) : "Tunis", 28...
- Les accolades { } = "ceci est un objet / un element".
- Le texte va entre "guillemets", les nombres non.

Une vraie API meteo repond avec un gros JSON contenant des donnees imbriquees. "Extraire" = aller chercher seulement ce dont on a besoin :

```
{
  "name": "Tunis",
  "main": { "temp": 28, "humidity": 45 },
  "weather": [ { "description": "clear sky" } ]
}
```

La syntaxe a retenir : le point (.)

- **main.temp** = "va dans main, puis prends temp" -> 28.
- **weather[0].description** = "va dans weather, prends le 1er element [0], puis description".
- Le [0] car weather est une **liste** (crochets []). Les listes commencent a 0, pas 1.

Regle facile : crochets [] -> c'est une liste -> il faut [0]. Accolades { } seules -> un objet -> juste le point (.).

Question : Avec le JSON { "name":"Sfax", "main":{"temp":31}, "weather":[{"description":"sunny"}] }, quel chemin pour la temperature (31) et pour la description ?

Reponse correcte : Temperature = **main.temp**. Description = **weather[0].description** (ne pas oublier le [0] car weather est une liste).

Lecon 4 - Comment les nodes se passent la data

Regle d'or : chaque node peut utiliser la data de n'importe quel node qui vient avant lui. La data ne disparaît pas, elle continue de circuler et les nodes suivants peuvent aller la reprendre.

Exemple : le node Send Email vient après HTTP Request, donc il peut utiliser les données météo dans le texte de l'email. C'est comme ça que l'email peut dire "Temperature : 28C".

Les expressions : {{ }}

Pour insérer une donnée dynamique (qui change chaque jour) dans un champ, on utilise une expression, écrite avec des doubles accolades :

```
Good morning!
Weather in {{ $json.name }} today:
Temperature: {{ $json.main.temp }}C
Condition: {{ $json.weather[0].description }}
```

- **{{ }}** = "n8n, ce n'est pas du texte normal, va chercher une vraie donnée ici".
- **\$json** = "la data qui vient du node précédent".
- **\$json.main.temp** = même règle du point que la Leçon 3.

Deux modes pour un champ : **Fixed** (texte fixe qui ne change jamais) ou **Expression** (avec {{ }} pour une donnée qui change à chaque exécution).

Question : Le JSON est { "name": "Tunis", "main": { "temp": 28 } }. Comment écrire l'expression pour insérer le nom de la ville dans l'email ?

Réponse correcte : **{{ \$json.name }}** - à l'exécution, l'email affichera "Tunis".

Lecon 5 - Credentials & cles API (securite)

Deux services de ton projet ont besoin d'une preuve d'identite avant de te laisser entrer : l'API meteo, et ton compte email.

1. La cle API = ton mot de passe personnel pour un service

- Il faut s'inscrire (compte gratuit) pour obtenir une cle API : une longue chaine secrete.
- C'est comme un ticket ou une carte de membre. A chaque appel, le node montre la cle et l'API repond.
- Pour ce projet : OpenWeatherMap (gratuit, largement suffisant). Inscription une fois, on copie la cle.
- Important : la cle API est SECRETE. Ne jamais la partager ni la mettre dans une capture d'ecran.

2. Les Credentials = le coffre-fort de n8n

- On entre la cle / le login une seule fois, dans un Credential.
- n8n le chiffre et le stocke en securite.
- Ensuite chaque node choisit simplement ce credential dans une liste, sans retaper le secret.

401	Unauthorized - "Qui es-tu ?"	Cle API absente ou fausse (probleme d'identite)
404	Not Found - "cette adresse n'existe pas"	Mauvaise URL / lien (probleme d'adresse)

Question : Si le node HTTP Request appelle l'API sans la cle API, que se passe-t-il et pourquoi ?

Reponse correcte : L'API refuse et renvoie une erreur **401 Unauthorized** au lieu des donnees, parce qu'elle ne sait pas qui tu es (pas de preuve d'identite).

Recapitulatif - Toute la theorie en un coup d'oeil

Lecon	A retenir
1. C'est quoi n8n	Une chaine de montage : workflow = la chaine, node = une machine, la data circule de gauche a droite.
2. Types de nodes	3 familles : Trigger (quand ca démarre), Action (faire/API), Logic-Data (decider/nettoyer).
3. Le JSON	Reponse des API en paires cle:valeur. Le point (.) pour entrer, [0] pour les listes.
4. Expressions	Les nodes reutilisent la data avec {{ \$json.main.temp }}.
5. Cles & Credentials	Cle API = ticket secret. Credentials = coffre-fort de n8n. Pas de cle -> erreur 401.

Prochaine etape : LA PRATIQUE - creer le compte n8n, le compte OpenWeatherMap, et construire le workflow ensemble (Schedule -> HTTP Request -> Edit Fields -> Send Email), puis ajouter des ameliorations.